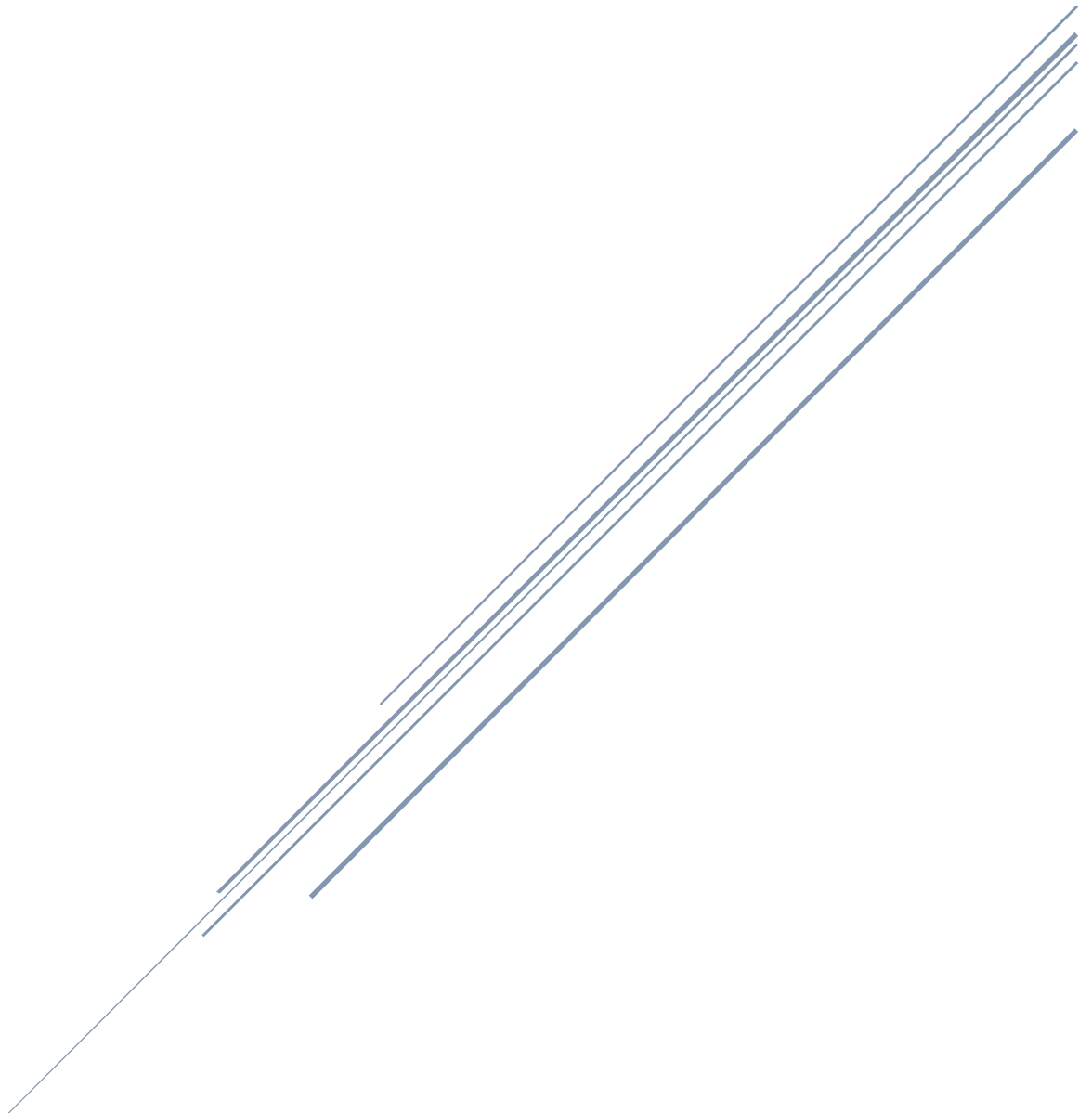


ENTERPRISE ANALYTICS & REPORTING LAYER

AdventureWorks Database



Saira Fatima
DevSquad '26 Internship NetixSol

Contents

1. Introduction.....	2
2. Project Objectives	2
3. Problem Statement	2
4. Database Overview	3
5. Business Domains Covered	3
6. Analytics Architecture	4
7. SQL Pipeline Design.....	4
Stage 1 – Foundation Layer	4
Stage 2 – Domain Analytics.....	4
Stage 3 – Business Metrics	4
Stage 4 – Executive Reporting.....	5
Stage 5 – Python Dashboard	5
8. Reusable Analytical Views	5
9. Advanced SQL Techniques	5
10. Python Analytics Notebook	5
11. Dashboard Visualizations	6
12. Executive KPI Summary.....	6
13. Business Insights.....	7
14. Executive Recommendations.....	7
15. Challenges Faced.....	7
16. Design Decisions	7
17. Assumptions.....	7
18. Future Improvements	8
19. Conclusion	8
20. Project Structure	8

1. Introduction

Modern organizations generate vast amounts of transactional data every day through sales, purchasing, inventory management, customer interactions, and employee operations. While transactional databases are optimized for efficiently processing daily business activities (OLTP), they are not suitable for analytical reporting because business intelligence requires repeated aggregations, complex joins, and historical trend analysis.

To overcome these limitations, organizations build an intermediate analytics layer that transforms raw operational data into reusable business-ready datasets.

This project implements a complete Enterprise Analytics & Reporting Layer on top of the AdventureWorks PostgreSQL database. Instead of repeatedly querying operational tables, reusable analytical views were created inside a dedicated analytics schema. These views provide a centralized, consistent, and scalable reporting layer that powers executive dashboards and future business intelligence applications.

2. Project Objectives

The primary objectives of this project are:

- Design a reusable analytics layer.
- Transform operational data into business-ready datasets.
- Build a dependency-based SQL pipeline.
- Demonstrate advanced SQL concepts.
- Generate executive dashboards using Python.
- Produce actionable business recommendations.
- Ensure future dashboards query only analytical views instead of raw tables.

3. Problem Statement

The AdventureWorks database is designed for transactional processing rather than analytical reporting.

Business users often require information such as:

- Monthly revenue trends
- Quarterly revenue
- Customer lifetime value
- Product profitability
- Territory performance
- Employee rankings
- Inventory health

Running these analytical queries directly on operational tables leads to:

- Slow query execution
- Repeated calculations
- Duplicate business logic
- Poor maintainability
- Inconsistent reporting

This project solves these challenges by introducing a reusable analytics layer.

4. Database Overview

The AdventureWorks database contains data distributed across multiple enterprise schemas.

The project integrates data from:

- Sales
- Production
- Purchasing
- HumanResources
- Person

These schemas collectively represent the complete operational workflow of an enterprise.

Because these tables are highly normalized, reporting queries require multiple joins and aggregations. Therefore, analytical views are created to simplify reporting and improve performance.

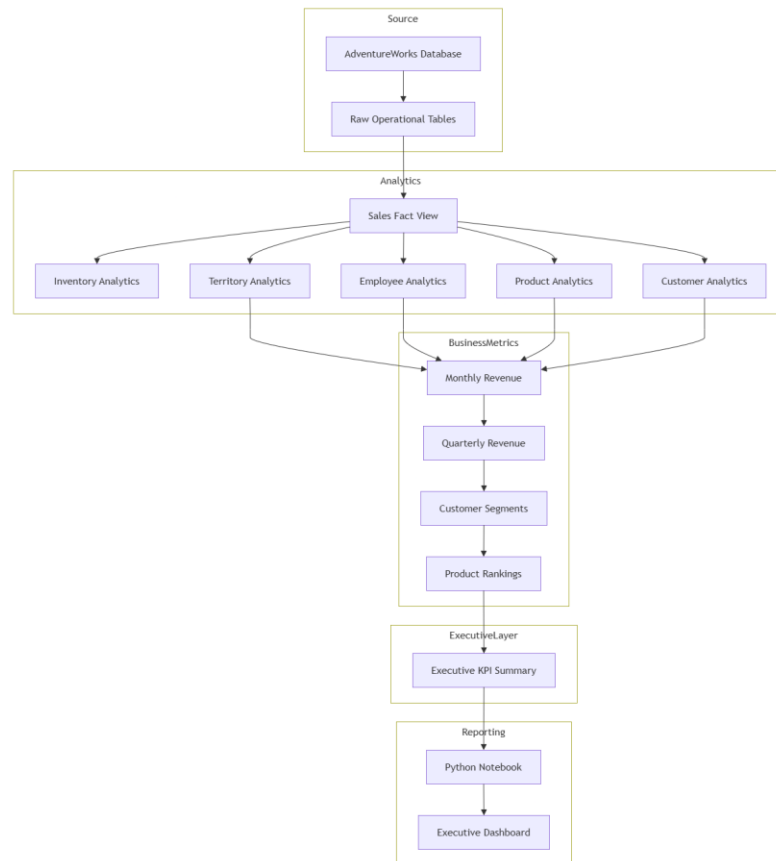
5. Business Domains Covered

The analytics layer integrates seven business domains.

Business Domain	Purpose
Sales	Revenue and order analysis
Customers	Customer behavior and segmentation
Products	Product profitability and rankings
Employees	Salesperson performance
Territories	Regional analysis
Inventory	Stock monitoring
Purchasing	Supplier and purchasing analysis

This satisfies the assignment requirement of using at least five business domains.

5. Analytics Architecture



7. SQL Pipeline Design

The SQL implementation follows a chained dependency model.

Stage 1 – Foundation Layer

- `vw_sales_fact`

Stage 2 – Domain Analytics

- `vw_customer_analytics`
- `vw_product_analytics`
- `vw_employee_analytics`
- `vw_territory_analytics`
- `vw_inventory_purchasing_analytics`

Stage 3 – Business Metrics

- Monthly Revenue
- Quarterly Revenue
- Customer Segments
- Product Rankings

Stage 4 – Executive Reporting

- Executive KPI Summary

Stage 5 – Python Dashboard

8. Reusable Analytical Views

View	Description
vw_sales_fact	Unified sales fact table
vw_customer_analytics	Customer metrics
vw_product_analytics	Product profitability
vw_monthly_revenue	Monthly trends
vw_quarterly_revenue	Quarterly trends
vw_customer_segments	RFM segmentation
vw_product_rankings	Product rankings
vw_employee_analytics	Employee KPIs
vw_territory_analytics	Territory KPIs
vw_inventory_purchasing_analytics	Inventory health
vw_executive_kpi_summary	Executive dashboard KPIs

9. Advanced SQL Techniques

The project demonstrates several advanced SQL concepts.

- Common Table Expressions (CTEs)
- Window Functions
- LAG()
- RANK()
- DENSE_RANK()
- CASE WHEN
- Conditional Aggregation
- INNER JOIN
- LEFT JOIN
- Ranking Functions
- Reusable Views
- Chained SQL Pipeline
- Business Metric Calculations

These techniques ensure maintainability, scalability, and reusable analytics.

10. Python Analytics Notebook

The notebook serves as the reporting layer.

Key features include:

- Secure PostgreSQL connection using .env
- SQLAlchemy integration
- `pandas.read_sql()` for reading analytical views
- No direct access to raw operational tables
- Executive dashboards generated from reusable analytics views

11. Dashboard Visualizations

The notebook contains executive-level visualizations generated exclusively from the analytics layer.

Visualizations include:

- Monthly Revenue Trend
- Quarterly Revenue
- Sales by Territory
- Customer Segments
- Product Performance
- Category Revenue
- Employee Performance
- Inventory Status
- Executive KPI Summary

Each visualization includes a short business insight describing its significance.

12. Executive KPI Summary

The executive dashboard provides a consolidated overview of business performance.

Key indicators include:

- Total Company Revenue
- Total Company Margin
- Active Customers
- Champion Customers
- At-Risk Customers
- Top Territory
- Best Selling Product
- Average Month-over-Month Growth

These KPIs allow executives to quickly assess overall organizational performance.

13. Business Insights

The analysis identified several significant findings:

- Revenue is primarily driven by the Bikes category.
- Southwest consistently generates the highest revenue.
- Many customers belong to the At-Risk segment, indicating retention opportunities.
- High-revenue products do not always produce high profit margins.
- Sales performance is concentrated among a limited number of representatives.

14. Executive Recommendations

Include your finalized:

- Five Business Opportunities
- Five Business Risks
- Five Actionable Recommendations

15. Challenges Faced

- Complex schema relationships across multiple business domains.
- Secure database credential management using `.env`.
- Time-series analytics requiring date conversion for quarterly aggregation.
- Eliminating redundant calculations through reusable analytical views.

16. Design Decisions

Key architectural decisions include:

- Dedicated analytics schema
- Modular reusable views
- Chained SQL pipeline
- Business logic centralized within SQL
- Python querying only analytical views

17. Assumptions

- Revenue is based on completed sales transactions.
- Partial data at the end of the reporting period reflects data availability rather than business decline.

- Analytical views represent the authoritative reporting layer.

18. Future Improvements

Potential enhancements include:

- Incremental refresh strategies
- Materialized views for large datasets
- Interactive dashboards using Power BI or Tableau
- Automated ETL scheduling
- Predictive sales forecasting using machine learning

19. Conclusion

The project successfully transforms the AdventureWorks transactional database into a reusable analytics platform capable of supporting executive reporting. By separating analytical workloads from operational tables, the solution improves scalability, maintainability, consistency, and reporting performance. The layered architecture ensures that future dashboards can be developed by querying only reusable analytical views, aligning with modern analytics engineering practices.

20. Project Structure

Week3/

└─ day5/

│ └─ README.md

│ └─ analytics_pipeline.sql

│ └─ executive_analysis.ipynb

│ └─ charts/

│ └─ screenshots/

└─ documentation/

└─ Project_Documentation.pdf